

Design of correct by construction algorithms by double checking properties

Dominique Méry
Telecom Nancy, Université de Lorraine
dominique.mery@loria.fr

**Egon Börger 80th Birthday Festschrift
Meeting in Pisa, Italy, 15 - 16 June 2026**

Summary

- 1 Overview
- 2 Programming by contract
 - Short view
 - The call-as-event idea
 - Main idea
- 3 Short Summary on Event-B
- 4 Internalizing events by merging
- 5 Summing a list of integer values
- 6 Summary - conclusion - future works

- 1 Overview
- 2 Programming by contract
 - Short view
 - The call-as-event idea
 - Main idea
- 3 Short Summary on Event-B
- 4 Internalizing events by merging
- 5 Summing a list of integer values
- 6 Summary - conclusion - future works

Introduction

- Guidelines for using the transformations proposed by J.-R. Abrial and used for generating sequential algorithms.

- Guidelines for using the transformations proposed by J.-R. Abrial and used for generating sequential algorithms.
- Possibility to combine with others provers as TLAPS, Frama-c

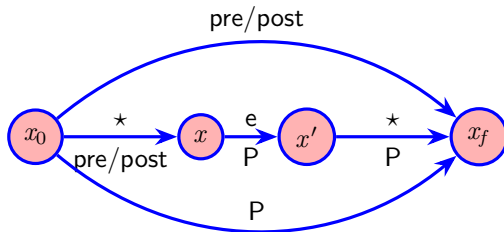
- Guidelines for using the transformations proposed by J.-R. Abrial and used for generating sequential algorithms.
- Possibility to combine with others provers as TLAPS, Frama-c
- Illustrating the merging of events and the step to code.

- Guidelines for using the transformations proposed by J.-R. Abrial and used for generating sequential algorithms.
- Possibility to combine with others provers as TLAPS, Frama-c
- Illustrating the merging of events and the step to code.
- Relationship to PlusCal and TLA⁺ proof tools

Big steps versus Small steps

From pre/post to algorithm(s)

Refining a pre/post specification into a sequential algorithm



Current Summary

- 1 Overview
- 2 Programming by contract
 - Short view
 - The call-as-event idea
 - Main idea
- 3 Short Summary on Event-B
- 4 Internalizing events by merging
- 5 Summing a list of integer values
- 6 Summary - conclusion - future works

Current Summary

- 1 Overview
- 2 Programming by contract
 - Short view
 - The call-as-event idea
 - Main idea
- 3 Short Summary on Event-B
- 4 Internalizing events by merging
- 5 Summing a list of integer values
- 6 Summary - conclusion - future works

Verification of contract (I)

A program P *satisfies* a contract $(pre, post)$:

- P transforms a variable x from an initial value x_0 and produces a final value x_f : $x_0 \xrightarrow{P} x_f$
- x_0 satisfies pre : $pre(x_0)$ and x_f satisfies $post$: $post(x_0, x_f)$
- $pre(x_0) \wedge x_0 \xrightarrow{P} x_f \Rightarrow post(x_0, x_f)$

requires $pre(x_0)$
ensures $post(x_0, x_f)$
variables X

begin
 $0 : P_0(x_0, x)$
instruction₀
...
 $i : P_i(x_0, x)$
...
instruction _{$f-1$}
 $f : P_f(x_0, x)$
end

Floyd style

- $pre(x_0) \wedge x = x_0 \Rightarrow P_0(x_0, x)$
- $pre(x_0) \wedge P_f(x_0, x) \Rightarrow post(x_0, x)$
- For each pair ℓ, ℓ'
such that $\ell \longrightarrow \ell'$, one checks that
for any value $x, x' \in \text{MEMORY}$
$$\left(\begin{array}{l} pre(x_0) \wedge P_\ell(x_0, x) \\ \wedge cond_{\ell, \ell'}(x) \wedge x' = f_{\ell, \ell'}(x) \end{array} \right) \Rightarrow P_{\ell'}(x_0, x')$$

Verification du contract (II)

A program P *satisfies* a contract $(pre, post)$:

- P transforms a variable x from an initial value x_0 and produces a final value x_f : $x_0 \xrightarrow{P} x_f$
- x_0 satisfait pre : $pre(x_0)$ and x_f satisfait $post$: $post(x_0, x_f)$
- $pre(x_0) \wedge x_0 \xrightarrow{P} x_f \Rightarrow post(x_0, x_f)$

requires $pre(x_0)$
ensures $post(x_0, x_f)$
variables X

```
begin  
  0 :  $P_0(x_0, x)$   
  instruction0  
  ...  
  i :  $P_i(x_0, x)$   
  ...  
  instructionf-1  
  f :  $P_f(x_0, x)$   
end
```

Hoare style

Verification du contract (II)

A program P *satisfies* a contract $(\text{pre}, \text{post})$:

- P transforms a variable x from an initial value x_0 and produces a final value x_f : $x_0 \xrightarrow{P} x_f$
- x_0 satisfait pre : $\text{pre}(x_0)$ and x_f satisfait post : $\text{post}(x_0, x_f)$
- $\text{pre}(x_0) \wedge x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f)$

requires $\text{pre}(x_0)$

ensures $\text{post}(x_0, x_f)$

variables X

begin

$0 : P_0(x_0, x)$

instruction₀

...

$i : P_i(x_0, x)$

...

instruction _{$f-1$}

$f : P_f(x_0, x)$

end

Hoare style

- $\forall x_f, x_0. \text{pre}(x_0) \wedge x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f)$

Verification du contract (II)

A program P *satisfies* a contract $(\text{pre}, \text{post})$:

- P transforms a variable x from an initial value x_0 and produces a final value x_f : $x_0 \xrightarrow{P} x_f$
- x_0 satisfait pre : $\text{pre}(x_0)$ and x_f satisfait post : $\text{post}(x_0, x_f)$
- $\text{pre}(x_0) \wedge x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f)$

requires $\text{pre}(x_0)$

ensures $\text{post}(x_0, x_f)$

variables X

begin

$0 : P_0(x_0, x)$

instruction₀

...

$i : P_i(x_0, x)$

...

instruction _{$f-1$}

$f : P_f(x_0, x)$

end

Hoare style

- $\forall x_f, x_0. \text{pre}(x_0) \wedge x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f)$
- $\forall x_f, x_0. \text{pre}(x_0) \Rightarrow (x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f))$

Verification du contract (II)

A program P *satisfies* a contract $(\text{pre}, \text{post})$:

- P transforms a variable x from an initial value x_0 and produces a final value x_f : $x_0 \xrightarrow{P} x_f$
- x_0 satisfait pre : $\text{pre}(x_0)$ and x_f satisfait post : $\text{post}(x_0, x_f)$
- $\text{pre}(x_0) \wedge x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f)$

requires $\text{pre}(x_0)$

ensures $\text{post}(x_0, x_f)$

variables X

begin

$0 : P_0(x_0, x)$

instruction₀

...

$i : P_i(x_0, x)$

...

instruction _{$f-1$}

$f : P_f(x_0, x)$

end

Hoare style

- $\forall x_f, x_0. \text{pre}(x_0) \wedge x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f)$
- $\forall x_f, x_0. \text{pre}(x_0) \Rightarrow (x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f))$
- $\forall x_0. \text{pre}(x_0) \Rightarrow \forall x_f. (x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f))$

Verification du contract (II)

A program P *satisfies* a contract (pre,post):

- P transforms a variable x from an initial value x_0 and produces a final value x_f : $x_0 \xrightarrow{P} x_f$
- x_0 satisfait pre: $\text{pre}(x_0)$ and x_f satisfait post : $\text{post}(x_0, x_f)$
- $\text{pre}(x_0) \wedge x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f)$

requires $\text{pre}(x_0)$

ensures $\text{post}(x_0, x_f)$

variables X

begin

$0 : P_0(x_0, x)$

instruction₀

...

$i : P_i(x_0, x)$

...

instruction _{$f-1$}

$f : P_f(x_0, x)$

end

Hoare style

- $\forall x_f, x_0. \text{pre}(x_0) \wedge x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f)$
- $\forall x_f, x_0. \text{pre}(x_0) \Rightarrow (x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f))$
- $\forall x_0. \text{pre}(x_0) \Rightarrow \forall x_f. (x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f))$
- $\forall x_0. \text{pre}(x_0) \Rightarrow \forall x. (x_0 \xrightarrow{P} x \Rightarrow \text{post}(x_0, x))$

Verification du contract (II)

A program P *satisfies* a contract $(\text{pre}, \text{post})$:

- P transforms a variable x from an initial value x_0 and produces a final value x_f : $x_0 \xrightarrow{P} x_f$
- x_0 satisfait pre : $\text{pre}(x_0)$ and x_f satisfait post : $\text{post}(x_0, x_f)$
- $\text{pre}(x_0) \wedge x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f)$

requires $\text{pre}(x_0)$

ensures $\text{post}(x_0, x_f)$

variables X

begin

$0 : P_0(x_0, x)$

instruction₀

...

$i : P_i(x_0, x)$

...

instruction _{$f-1$}

$f : P_f(x_0, x)$

end

Hoare style

- $\forall x_f, x_0. \text{pre}(x_0) \wedge x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f)$
- $\forall x_f, x_0. \text{pre}(x_0) \Rightarrow (x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f))$
- $\forall x_0. \text{pre}(x_0) \Rightarrow \forall x_f. (x_0 \xrightarrow{P} x_f \Rightarrow \text{post}(x_0, x_f))$
- $\forall x_0. \text{pre}(x_0) \Rightarrow \forall x. (x_0 \xrightarrow{P} x \Rightarrow \text{post}(x_0, x))$
- $\forall x_0. \text{pre}(x_0) \Rightarrow WLP(P)(\text{post}(x_0, x))$

A Simple C Function

Listing 1: Simple contract

```
#ifndef _A_H
#define _A_H
// Definition of the mathematical function mathpower2
/*@ axiomatic auxmath {
  @ lemma rule1: \forall int n; n > 0 ==> n*n == (n-1)*(n-1)+2*(n-1)+1;
  @ } */
/*@ requires 0 <= x;
    @ requires x*x <= INT_MAX;
    assigns \nothing;
    ensures \result == x*x;
*/
int power2(int x);
#endif
```

A Simple C Function

Listing 2: Simple contract

```
#include <limits.h>
#include "power2.h"

int power2(int x)
{
    int r, k, cv, cw, or, ok, ocv, ocw;
    r=0; k=0; cv=0; cw=0; or=0; ok=k; ocv=cv; ocw=cw;
    /*@ loop invariant 0 <= cv && 0 <= cw && 0 <= k;
       @ loop invariant cv == k*k;
       @ loop invariant k <= x;
       @ loop invariant cw == 2*k;
       @ loop invariant 4*cv == cw*cw;
       @ loop assigns k, cv, cw, or, ok, ocv, ocw;
       @ loop variant x-k;
    */
    while (k<x)
    {
        ok=k; ocv=cv; ocw=cw;
        k=ok+1;
        cv=ocv+ocw+1;
        cw=ocw+2;
    }
    r=cv;
    return(r);
}
```

A Simple C Function

Listing 3: Simple contract

```
frama-c -wp -rte power2.c
[kernel] Parsing power2.c (with preprocessing)
[rte:annot] annotating function power2
[wp] 30 goals scheduled
[wp] Proved goals: 32 / 32
    Terminating: 1
    Unreachable: 1
    Qed: 22 (5ms—1ms—7ms)
    Alt—Ergo 2.6.0: 8 (13ms—15ms—18ms)
```

- Finding or computing annotations is difficult and even undecidable!

- Finding or computing annotations is difficult and even undecidable!

- Given

Contract EX
Variables $X(int)$
Requires $pre(x_0)$
Ensures $post(x_0, x_f)$

Correct by Construction

- Finding or computing annotations is difficult and even undecidable!

- Given

Contract EX
Variables $X(int)$
Requires $pre(x_0)$
Ensures $post(x_0, x_f)$

- Design of the algorithm ALG fulfilling the contract

- Finding or computing annotations is difficult and even undecidable!

- Given

Contract EX
Variables $X(int)$
Requires $pre(x_0)$
Ensures $post(x_0, x_f)$

- Design of the algorithm ALG fulfilling the contract
- HOARE triple:

$$\{\mathbf{pre}(\mathbf{x}_0) \wedge \mathbf{x} = \mathbf{x}_0\} \text{ALG} \{\mathbf{post}(\mathbf{x}_0, \mathbf{x})\}$$

- Finding or computing annotations is difficult and even undecidable!

- Given

Contract EX Variables $X(int)$ Requires $pre(x_0)$ Ensures $post(x_0, x_f)$
--

- Design of the algorithm ALG fulfilling the contract
- HOARE triple:

$$\{\mathbf{pre}(\mathbf{x}_0) \wedge \mathbf{x} = \mathbf{x}_0\} \text{ALG} \{\mathbf{post}(\mathbf{x}_0, \mathbf{x})\}$$

Idea?

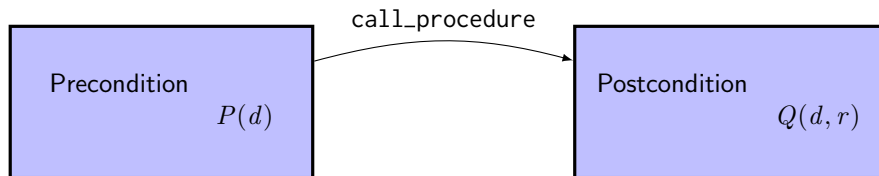
From Contract EX , using a step by step approach to find an algorithm ALG satisfying it using refinement and Event-B models.
Applying the CLEANROOM model but with tool support

Current Summary

- 1 Overview
- 2 Programming by contract
 - Short view
 - The call-as-event idea**
 - Main idea
- 3 Short Summary on Event-B
- 4 Internalizing events by merging
- 5 Summing a list of integer values
- 6 Summary - conclusion - future works

- Providing patterns for helping the use of refinement.
- Making proofs as simple as possible.
- Simplifying the discovery of invariants
- we focus on the transformation of the final concrete specification into a PlusCal algorithm:
 - ① transforming an `EVENT B` specification into a PlusCal algorithm
 - ② using the merging of events to get only one event corresponding to the call of the algorithm
 - ③ double checking the resulting algorithm by using TLA^+ and its proof system.

The call as event idea



- `call_procedure` is simulated by the event **call_procedure**:

```
EVENT  CALL_PROCEDURE
When
   $P(d)$ 
Then
   $r : | Q(d, r')$ 
end
```

- $P(d)$ is specifying the inputs of the procedure.
- $Q(d, r)$ is expressing the relation between the inputs d and the results r .

Current Summary

- 1 Overview
- 2 Programming by contract
 - Short view
 - The call-as-event idea
 - Main idea
- 3 Short Summary on Event-B
- 4 Internalizing events by merging
- 5 Summing a list of integer values
- 6 Summary - conclusion - future works

Call as event

- A pre/post specification is a pair of assertions $(pre, post)$
- $(pre, post)$ is defining a *transformation* over variables that is correct with respect to *HOARE* triples:

$$\{pre\} transformation \{post\}$$

- **Problem:** Design of a program PROG simulating the *transformation* with respect to $(pre, post)$:

$$\{pre\} PROG \{post\}$$

- **Idea:** Writing a *one-shot* relation simulating the relation over *flexible* variables:

$$\forall x, y. pre(x) \wedge R(x, y) \Rightarrow post(x, y)$$

- **event:** Expressing the relation:

EVENT **PROG**

When

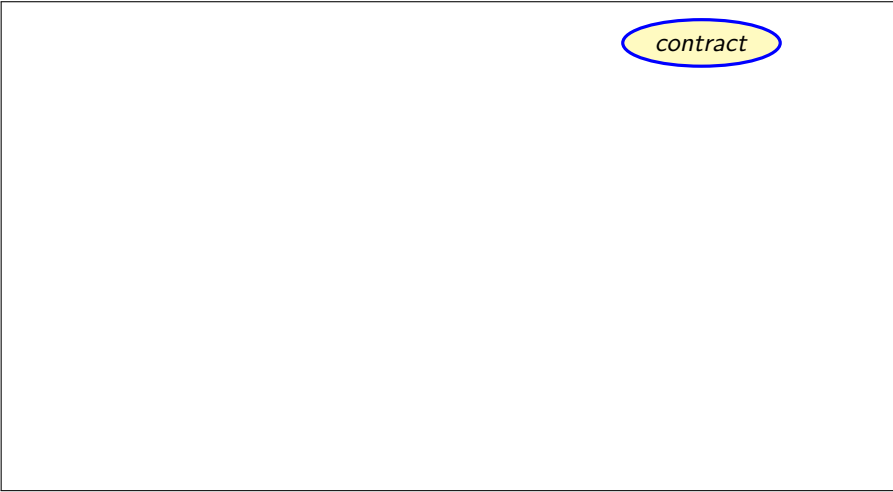
$pre(x)$

Then

$y : |post(x, y')$

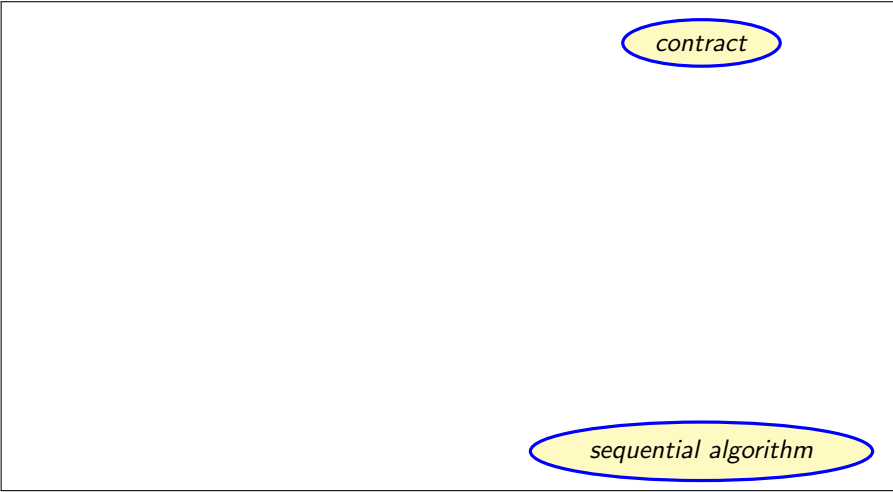
end

The sequential pattern



contract

The sequential pattern

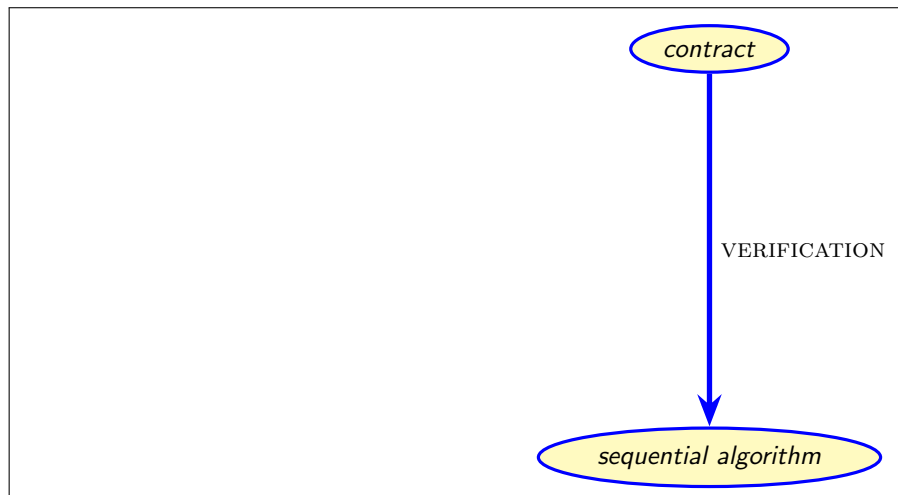


A diagram illustrating the sequential pattern. It consists of a large light gray rectangle with a black border. Inside this rectangle, there are two yellow ovals with blue outlines. The top oval is located in the upper right area and contains the word "contract". The bottom oval is located in the lower right area and contains the words "sequential algorithm".

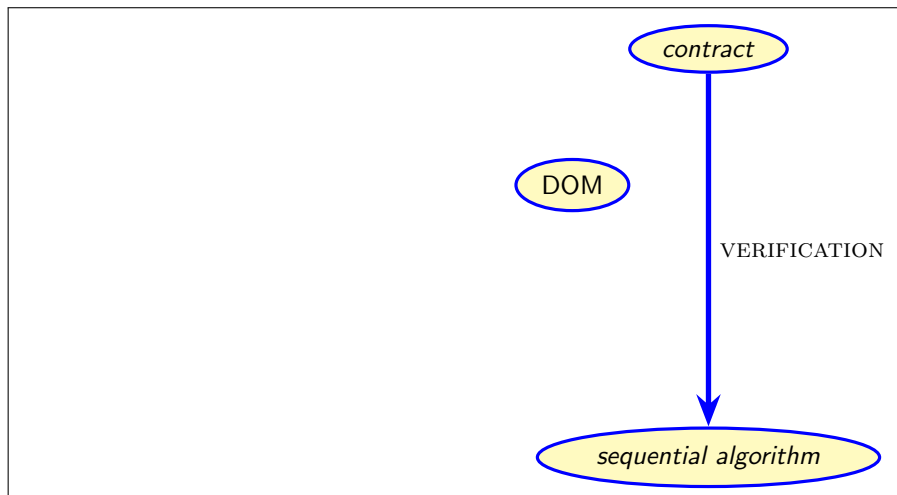
contract

sequential algorithm

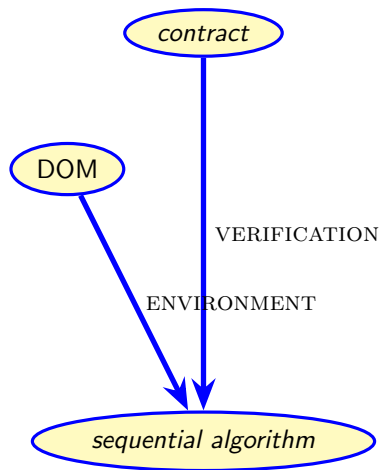
The sequential pattern



The sequential pattern

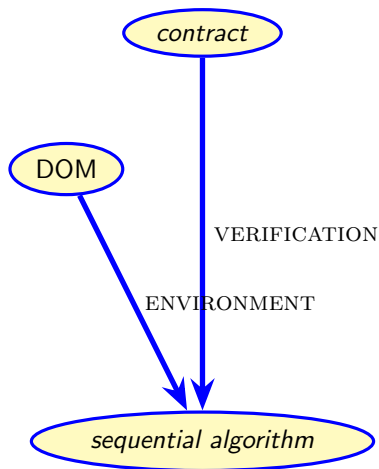


The sequential pattern

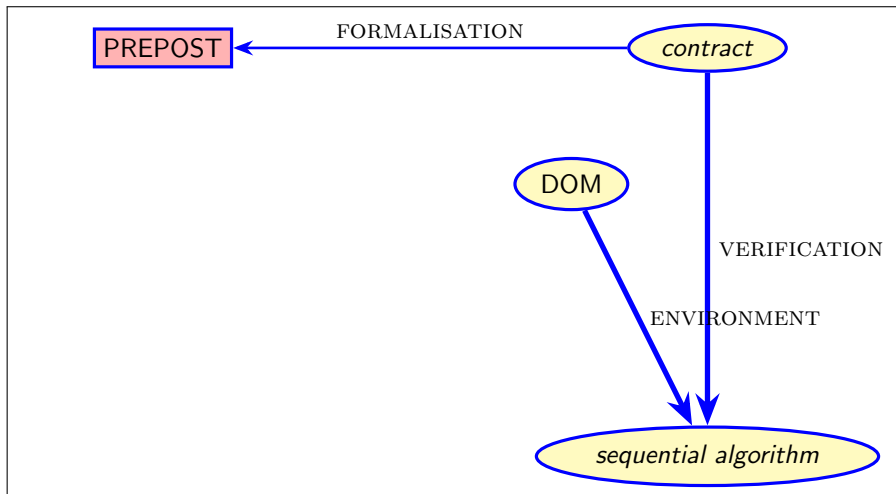


The sequential pattern

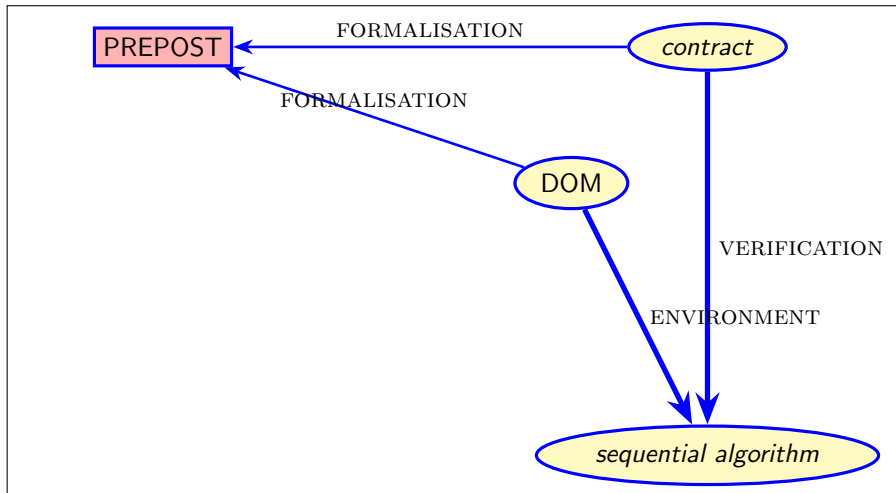
PREPOST



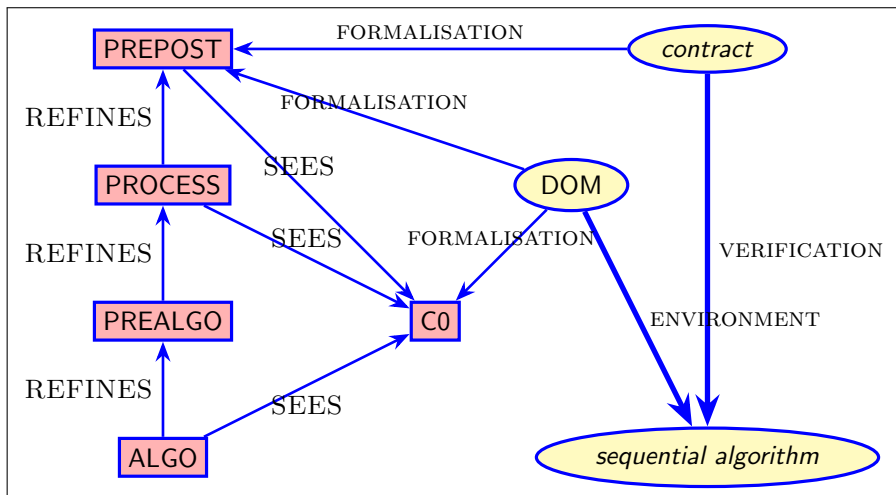
The sequential pattern



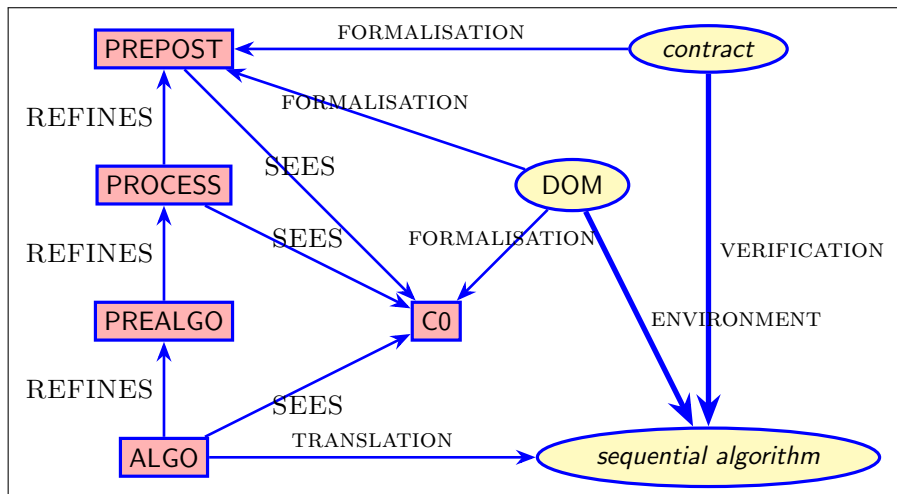
The sequential pattern



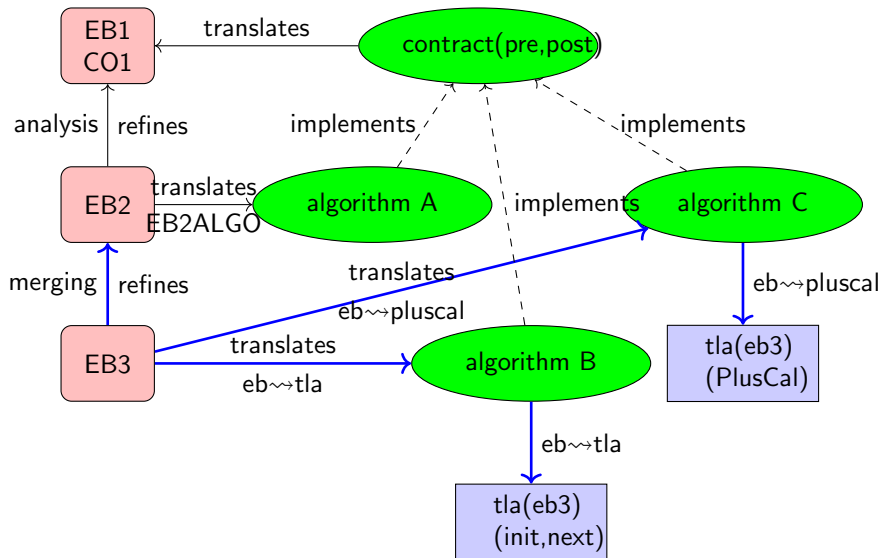
The sequential pattern



The sequential pattern



Global Picture



Current Summary

- 1 Overview
- 2 Programming by contract
 - Short view
 - The call-as-event idea
 - Main idea
- 3 Short Summary on Event-B
- 4 Internalizing events by merging
- 5 Summing a list of integer values
- 6 Summary - conclusion - future works

Short Summary on Event-B (context)

- Context: static properties of Event-B models
 - ▶ Sets: user-defined types
 - ▶ Constants: static object in development
 - ▶ Axioms: presumed properties about sets and constants
 - ▶ Theorems: derived properties about sets and constants

SETS

A

CONSTANTS

B, C, f

AXIOMS

$ax1 : B \subseteq A$

$ax2 : C \subseteq A$

$ax3 : g \in B \leftrightarrow C$

$ax4 : \forall A. A \subseteq \mathbb{N} \wedge 0 \in A \wedge suc[A] \subseteq A \Rightarrow \mathbb{N} \subseteq A$

...

Short Summary on Event-B (discrete)

■ Machine: behavioral properties of Event-B models

- ▶ Variables: states
- ▶ Invariants: properties of variables that always need to hold
- ▶ Theorems: derived properties about variables
- ▶ Events: possible state changes

EVENT e

ANY t

WHERE

$G(c, s, t, x)$

THEN

$x : |(P(c, s, t, x, x'))$

END

- c et s are constantes and visible sets by e
- x is a state variable or a list of variables
- $G(c, s, t, x)$ is the condition for observing e .
- $P(c, s, t, x, x')$ is the assertion for the relation over x and x' .
- $BA(e)(c, s, x, x')$ is the *before-after* relationship for e and is defined by $\exists t. G(c, s, t, x) \wedge P(c, s, t, x, x')$.

Short Summary on Event-B (refinement)

- Given an **abstract** and a corresponding **concrete** event

EVENT **ae** $\hat{=}$
any v **where**
 $G(x, v)$
then
 $x := E(x, v)$
end

EVENT **ce** $\hat{=}$
any w **where**
 $H(y, w)$
then
 $y := F(y, w)$
end

$$\begin{aligned} I(x) \wedge J(x, y) \wedge H(y, w) \\ \implies \\ \exists v \cdot (G(x, v) \wedge J(E(x, v), F(y, w))) \end{aligned}$$

- $BA(ae)(x, x') \hat{=} \exists v. G(x, v) \wedge x' = E(x, v)$
- $BA(ce)(y, y') \hat{=} \exists w. H(y, w) \wedge y' = F(y, w)$

Modelling systems in Event-B

MACHINE

m

sees

c

variables

x

invariant

I(x)

theorems

Q(x)

initialisation

Init(x)

events

... e

end

Modelling systems in Event-B

MACHINE

m

sees

c

variables

x

invariant

$I(x)$

theorems

$Q(x)$

initialisation

$Init(x)$

events

$\dots e$

end

c defines the static environment for the proofs related to m : sets, constants, axioms, theorems $\Gamma(m)$.

Modelling systems in Event-B

MACHINE

m

sees

c

variables

x

invariant

I(x)

theorems

Q(x)

initialisation

Init(x)

events

... e

end

c defines the static environment for the proofs related to *m*: sets, constants, axioms, theorems $\Gamma(m)$.

$\Gamma(m) \vdash \forall x \in Values : \text{INIT}(x) \Rightarrow I(x)$

Modelling systems in Event-B

MACHINE

m

sees

c

variables

x

invariant

$I(x)$

theorems

$Q(x)$

initialisation

$Init(x)$

events

$\dots e$

end

c defines the static environment for the proofs related to m : sets, constants, axioms, theorems $\Gamma(m)$.

$\Gamma(m) \vdash \forall x \in Values : \text{INIT}(x) \Rightarrow I(x)$

$\forall e :$

$\Gamma(m) \vdash \forall x, x', u \in Values : I(x) \wedge R(u, x, x') \Rightarrow I(x')$

Modelling systems in Event-B

MACHINE

m

sees

c

variables

x

invariant

$I(x)$

theorems

$Q(x)$

initialisation

$Init(x)$

events

$\dots e$

end

c defines the static environment for the proofs related to m : sets, constants, axioms, theorems $\Gamma(m)$.

$\Gamma(m) \vdash \forall x \in Values : \text{INIT}(x) \Rightarrow I(x)$

$\forall e :$

$\Gamma(m) \vdash \forall x, x', u \in Values : I(x) \wedge R(u, x, x') \Rightarrow I(x')$

$\Gamma(m) \vdash \forall x \in Values : I(x) \Rightarrow Q(x)$

Modelling systems in Event-B

MACHINE

m

sees

c

variables

x

invariant

$I(x)$

theorems

$Q(x)$

initialisation

$Init(x)$

events

$\dots e$

end

c defines the static environment for the proofs related to m : sets, constants, axioms, theorems $\Gamma(m)$.

$\Gamma(m) \vdash \forall x \in Values : \text{INIT}(x) \Rightarrow I(x)$

$\forall e :$

$\Gamma(m) \vdash \forall x, x', u \in Values : I(x) \wedge R(u, x, x') \Rightarrow I(x')$

$\Gamma(m) \vdash \forall x \in Values : I(x) \Rightarrow Q(x)$

e

ANY

u

WHERE

$G(x, u)$

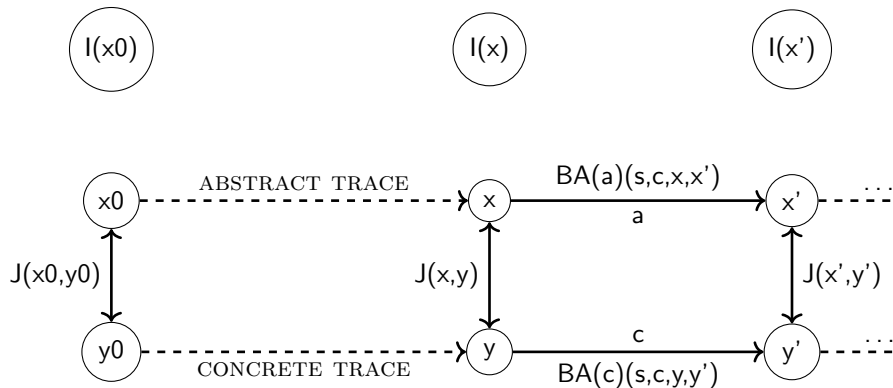
THEN

$x : |(R(u, x, x'))$

END

or e is **observed** $x \xrightarrow{e} x'$

Refinement between two machines



Current Summary

- 1 Overview
- 2 Programming by contract
 - Short view
 - The call-as-event idea
 - Main idea
- 3 Short Summary on Event-B
- 4 Internalizing events by merging
- 5 Summing a list of integer values
- 6 Summary - conclusion - future works

Merging two events

EVENT ae1

any u **where**

$G1(u, s, c, x)$

then

$x : |ABAP(u, s, c, x, x')$

end

EVENT ae2

any u **where**

$G2(u, s, c, x)$

then

$x : |ABAP(u, s, c, x, x')$

end

 **refines**

refines 

EVENT c

refines ae1, ae2

any u **where**

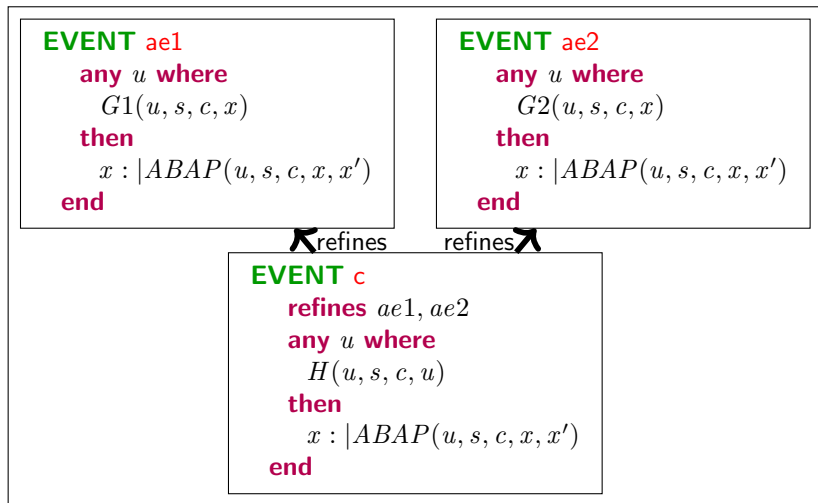
$H(u, s, c, u)$

then

$x : |ABAP(u, s, c, x, x')$

end

Merging two events



$AX(s, c), I(s, c, x), H(v, s, c, y) \vdash G_1(s, c, x) \vee G_2(s, c, x).$

Strategy for merging events

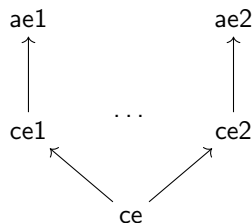
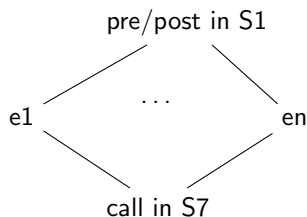
Let be two events $ae1$ and $ae2$ such that the two guards are complementary (ie $g(ae1) \Leftrightarrow \neg g(ae2)$ or $g(ae2) \Leftrightarrow \neg g(ae1)$).

- Let cei be the event with the guard $g(aei)$ and with the action $(g(ae1) \Rightarrow a(ae1)) \wedge (g(ae2) \Rightarrow a(ae2))$.

Then cei refines aei .

- Let ce be the event with the guard $g(ae1) \vee g(ae2)$.

Then ce refines cei by fusion.



Current Summary

- ① Overview
- ② Programming by contract
 - Short view
 - The call-as-event idea
 - Main idea
- ③ Short Summary on Event-B
- ④ Internalizing events by merging
- ⑤ Summing a list of integer values
- ⑥ Summary - conclusion - future works

Contract for the problem Summing a list of integer values

Contract for the problem Summing
a list of integer values

variables $n, v, r, done$

requires $\left\{ \begin{array}{l} n_0 \in \mathbb{N} \\ n_0 \neq 0 \\ v_0 \in 1..n_0 \rightarrow \mathbb{Z} \\ r_0 \in \mathbb{Z} \\ done_0 \in \mathbb{BOOL} \end{array} \right.$

ensures $\left(\begin{array}{l} r_f = \sum_{k=1}^{n_0} v_0(k) \\ done_f = TRUE \end{array} \right)$

Contract for the problem Summing a list of integer values

Contract for the problem Summing
a list of integer values

variables $n, v, r, done$

requires $\left\{ \begin{array}{l} n_0 \in \mathbb{N} \\ n_0 \neq 0 \\ v_0 \in 1..n_0 \rightarrow \mathbb{Z} \\ r_0 \in \mathbb{Z} \\ done_0 \in \text{BOOL} \end{array} \right.$

ensures $\left(\begin{array}{l} r_f = \sum_{k=1}^{n_0} v_0(k) \\ done_f = \text{TRUE} \end{array} \right.$

- the initial values of n , v and $done$, denoted n_0 , v_0 , $done_0$ and the final value of r denoted r_f .
- the final value of the variable r denoted r_f is the sum $\sum_{k=1}^{n_0} v_0(k)$.

Contract for the problem Summing a list of integer values

Contract for the problem Summing a list of integer values

variables $n, v, r, done$

requires $\left\{ \begin{array}{l} n_0 \in \mathbb{N} \\ n_0 \neq 0 \\ v_0 \in 1..n_0 \rightarrow \mathbb{Z} \\ r_0 \in \mathbb{Z} \\ done_0 \in \text{BOOL} \end{array} \right.$

ensures $\left(\begin{array}{l} r_f = \sum_{k=1}^{n_0} v_0(k) \\ done_f = \text{TRUE} \end{array} \right)$

- the initial values of n , v and $done$, denoted n_0 , v_0 , $done_0$ and the final value of r denoted r_f .
- the final value of the variable r denoted r_f is the sum $\sum_{k=1}^{n_0} v_0(k)$.

computation method

Defining the computation process by defining a sequence u in the context S_0

```
CONTEXT  S0
CONSTANTS  n0 v0 u
AXIOMS
@pre1 n0 ∈ ℕ
@pre2 n0 ≠ 0
@pre5 v0 ∈ 1..n0 → ℤ
@axm1 u ∈ 0..n0 → ℤ
@axm2 u(0) = 0
@axm3 ∀ k. k ∈ 1..n0
      ⇒ u(k) = u(k - 1) + v0(k)
end
```

```
CONTEXT S0
CONSTANTS n0 v0 u
AXIOMS
@pre1 n0 ∈ ℕ
@pre2 n0 ≠ 0
@pre5 v0 ∈ 1..n0 → ℤ
@axm1 u ∈ 0..n0 → ℤ
@axm2 u(0) = 0
@axm3 ∀ k. k ∈ 1..n0
      ⇒ u(k) = u(k - 1) + v0(k)
end
```

- The context S0 defines the sequence u supporting the computation method by definition of the summation expression.


```
CONTEXT S0
CONSTANTS n0 v0 u
AXIOMS
  @pre1 n0 ∈ ℕ
  @pre2 n0 ≠ 0
  @pre5 v0 ∈ 1..n0 → ℤ
  @axm1 u ∈ 0..n0 → ℤ
  @axm2 u(0) = 0
  @axm3 ∀ k. k ∈ 1..n0
           ⇒ u(k) = u(k - 1) + v0(k)
end
```

- The context S0 defines the sequence u supporting the computation method by definition of the summation expression.
- The refinement process starts by the specification namely S1 and leads to the machine S7 which is containing the event final internalizing the computation of the sum.

Expressing pre/post in a machine S1 (I)

```
MACHINE S1 SEES S0
VARIABLES r v n done
INVARIANTS
  @inv1 r ∈ ℤ
  @inv2 n ∈ ℤ
  @inv3 v ∈ 1..n → ℤ
  @inv4 done ∈ BOOL
  @read - values n = n0 ∧ v = v0
  @post done = TRUE
                    ⇒ r = u(n0)
EVENT INITIALISATION
  then
    @act1 r : ∈ ℤ
    @act2 n := n0
    @act3 v := v0
    @act4 done := FALSE
  end
```

- v and n are variables getting the input values as $v0$ and $n0$: *read-values* label.
- r is getting the result of the computing process under the control of *done*: *post* label.
- *post*: expression of the partial correctness.

Expressing pre/post in a machine S1 (II)

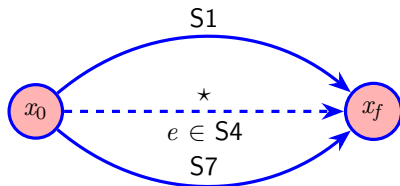
- The *final1* event expresses the relationship between initial and final values of variables.
- The *keep* event expresses the *hidden* loop preparing the final result.

```
EVENT final1
  where
    @grd1 done = FALSE
  then
    @act1 r := u(n)
    @act2 done := TRUE
  end

anticipated EVENT keep
  then
    @act1 r, n, v, done :|
      ( r' ∈  $\mathbb{Z}$  ∧ n' ∈  $\mathbb{Z}$ 
        ∧ v' ∈  $1..n' \rightarrow \mathbb{Z}$ 
        ∧ done' ∈ BOOL
        ∧ n' = n0 ∧ v' = v0
        ∧ (done' = TRUE
           ⇒ r' = u(n0)))
  end
```

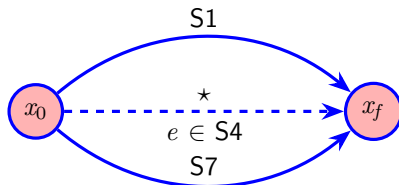
Summary of the refinement steps

Refinement from S1
till S7.



Summary of the refinement steps

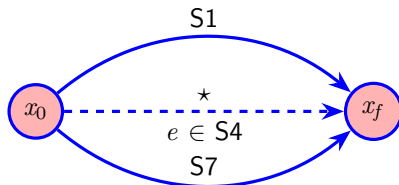
Refinement from S1
till S7.



- Identifying the final event as simulating the pre/post contract in S1.

Summary of the refinement steps

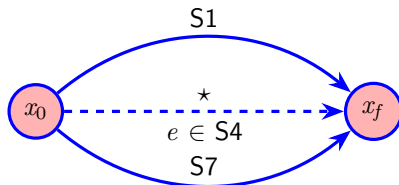
Refinement from S1
till S7.



- Identifying the final event as simulating the pre/post contract in S1.
- Refining by using u and introducing events simulating computing steps in S4.

Summary of the refinement steps

Refinement from S1
till S7.



- Identifying the final event as simulating the pre/post contract in S1.
- Refining by using u and introducing events simulating computing steps in S4.
- Merging events of S4 into one event in S7.

From S1 to S4 via S2 and S3

From S1 to S4 via S2 and S3

MACHINE *S2* REFINES *S1* SEES *S0*

VARIABLES *r i uu n v done*

INVARIANTS

@*inv1* $i \in 0..n$

@*inv2* $uu \in 0..n \leftrightarrow \mathbb{Z} \wedge \text{dom}(uu) = 0..i$

@*inv3* $\forall k. k \in \text{dom}(uu) \Rightarrow uu(k) = u(k)$

@*inv4* $done = FALSE \Rightarrow i \in 0..n$

@*inv6* $done = TRUE \Rightarrow i = n$

@*inv7* $i \in 0..n - 1 \Rightarrow done = FALSE$

theorem @*datath* $v = v0 \wedge n = n0$

VARIANT $n - i$

- The variable *uu* is recording the sequence *u* by computing until *i* is *n0*.
- The new event step is refining the event *keep* and decreases the variant.

From S1 to S4 via S2 and S3

MACHINE *S2* REFINES *S1* SEES *S0*

VARIABLES *r i uu n v done*

INVARIANTS

@inv1 $i \in 0..n$

@inv2 $uu \in 0..n \leftrightarrow \mathbb{Z} \wedge \text{dom}(uu) = 0..i$

@inv3 $\forall k. k \in \text{dom}(uu) \Rightarrow uu(k) = u(k)$

@inv4 $\text{done} = \text{FALSE} \Rightarrow i \in 0..n$

@inv6 $\text{done} = \text{TRUE} \Rightarrow i = n$

@inv7 $i \in 0..n-1 \Rightarrow \text{done} = \text{FALSE}$

theorem @datath $v = v0 \wedge n = n0$

VARIANT $n - i$

- The variable *uu* is recording the sequence *u* by computing until *i* is *n0*.
- The new event step is refining the event *keep* and decreases the variant.

MACHINE *S3* REFINES *S2* SEES *S0*

VARIABLES *r i uu cu n v done*

INVARIANTS

@inv1 $cu \in \mathbb{Z}$

@inv2 $cu = u(i)$

- The new variable *cu* is recording the last computed value of *uu*.
- Proofs of the invariants are made easier.

The machine S_4 in the middle (I)

```
MACHINE  $S_4$  REFINES  $S_3$  SEES  $S_0$ 
VARIABLES  $r\ i\ cu\ n\ v\ done$ 
INVARIANTS
  theorem @inv1  $cu = u(i)$ 
EVENTS
  EVENT INITIALISATION
    then
      @act1  $r : \in \mathbb{Z}$ 
      @act2  $i := 0$ 
      @act4  $cu := 0$ 
      @act5  $v := v_0$ 
      @act6  $n := n_0$ 
      @act7  $done := FALSE$ 
    end
```

The machine S4 in the middle (I)

```
MACHINE S4 REFINES S3 SEES S0
VARIABLES r i cu n v done
INVARIANTS
  theorem @inv1 cu = u(i)
EVENTS
  EVENT INITIALISATION
    then
      @act1 r : ∈ ℤ
      @act2 i := 0
      @act4 cu := 0
      @act5 v := v0
      @act6 n := n0
      @act7 done := FALSE
    end
```

- The variable uu helps in the proof process but is expensive.
- The variable uu is hidden in the abstractions.

The machine S4 in the middle (II)

```
EVENT final REFINES final
  where
    @grd1  $i = n$ 
    @grd2  $done = FALSE$ 
  then
    @act1  $r := cu$ 
    @act2  $done := TRUE$ 
end

EVENT step000 REFINES step
  where
    @grd1  $i < n$ 
    @grd2  $done = FALSE$ 
  then
    @act1  $i := i + 1$ 
    @act3  $cu := cu + v(i + 1)$ 
end
```

The machine S4 in the middle (II)

```
EVENT final REFINES final
  where
    @grd1  $i = n$ 
    @grd2  $done = FALSE$ 
    then
      @act1  $r := cu$ 
      @act2  $done := TRUE$ 
end

EVENT step000 REFINES step
  where
    @grd1  $i < n$ 
    @grd2  $done = FALSE$ 
    then
      @act1  $i := i + 1$ 
      @act3  $cu := cu + v(i + 1)$ 
end
```

- Events are made simpler and uu is hidden.
- Events are guarded by evaluable conditions as $i < n$ which is implying $i \notin dom(uu)$
- Using EB2ALGO, one gets an algorithm but no proof is given of the transformations.

The machine S7 with one event

MACHINE *S7* REFINES *S6* SEES *S0*

VARIABLES *r i cu n v done*

INVARIANTS

theorem @inv1 done = FALSE $\Rightarrow i \in 0..n$

theorem @inv2 done = TRUE $\Rightarrow i = n$

theorem @inv3 $i \in 0..n - 1 \Rightarrow done = FALSE$

theorem @inv4 $i \in 0..n$

EVENT *final* REFINES *final*

where

@grd2 done = FALSE

then

@act1 $r, i, cu, n, v, done :|$ (

($i = n$

$\Rightarrow (r' = cu \wedge cu' = cu \wedge i' = i \wedge n' = n$
 $\wedge v' = v \wedge done' = TRUE))$

$\wedge (i < n$

$\Rightarrow (r' = r \wedge cu' = cu + v(i + 1) \wedge i' = i + 1$
 $\wedge n' = n \wedge v' = v \wedge done' = done))$

)

end

Defining an algorithm by a set of actions

Defining an algorithm by a set of actions

$$\text{guard}(r, i, cu, n, v, done) \equiv done = FALSE$$
$$\text{action} \equiv \left[\begin{array}{l} i = n \Rightarrow \left[\begin{array}{l} (r' = cu \wedge cu' = cu \wedge i' = i \wedge n' = n \\ \wedge v' = v \wedge done' = TRUE) \end{array} \right. \\ i < n \Rightarrow \left[\begin{array}{l} r' = r \wedge cu' = cu + v(i + 1) \wedge i' = i + 1 \wedge n' = n \\ \wedge v' = v \wedge done' = done \end{array} \right. \end{array} \right.$$

Defining an algorithm by a set of actions

$$\text{guard}(r, i, cu, n, v, done) \equiv done = FALSE$$
$$\text{action} \equiv \left[\begin{array}{l} i = n \Rightarrow \left[\begin{array}{l} (r' = cu \wedge cu' = cu \wedge i' = i \wedge n' = n \\ \wedge v' = v \wedge done' = TRUE) \end{array} \right. \\ i < n \Rightarrow \left[\begin{array}{l} r' = r \wedge cu' = cu + v(i + 1) \wedge i' = i + 1 \wedge n' = n \\ \wedge v' = v \wedge done' = done \end{array} \right. \end{array} \right.$$

- The machine S7 contains an event Initialisation and a next relation



Defining an algorithm by a set of actions

$$\begin{aligned} \text{guard}(r, i, cu, n, v, done) &\equiv done = FALSE \\ \text{action} &\equiv \left[\begin{array}{l} i = n \Rightarrow \left[\begin{array}{l} (r' = cu \wedge cu' = cu \wedge i' = i \wedge n' = n \\ \wedge v' = v \wedge done' = TRUE) \end{array} \right. \\ i < n \Rightarrow \left[\begin{array}{l} r' = r \wedge cu' = cu + v(i + 1) \wedge i' = i + 1 \wedge n' = n \\ \wedge v' = v \wedge done' = done \end{array} \right. \end{array} \right. \end{aligned}$$

- The machine S7 contains an event Initialisation and a next relation
-

Definition

(mapping of a binary relation)

A binary relation R on $State$ can be mapped to a GKAT term e for the interpretation i , if $\mathcal{R}_i(e) = R$ and we write $map_i(R) = e$

Defining an algorithm by a set of actions (I)

Guarded Kleene Algebra with Tests

Summing a list of integer values is now considering the iteration property and we obtain

$$init.a^{(c)}b$$

where

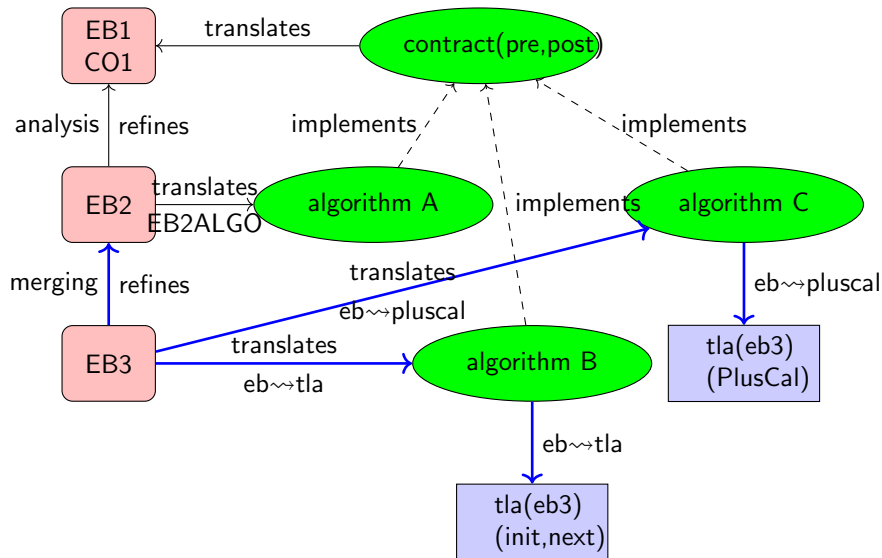
- $sat(init) = \{(n0, v0, FALSE, 0, 0, ?)\}$
- $\mathcal{R}_i(A)(s, s') \stackrel{def}{=} cu' = cu + v(i+1) \wedge i' = i+1 \wedge r' = r,$
 $a \stackrel{def}{=} cu := cu + v(i+1); i := i+1 \text{ and } map_i(A) = a$
- $\mathcal{R}_i(B)(s, s') \stackrel{def}{=} r' = cu \wedge done' = TRUE \wedge cu' = cu \wedge i' = i \text{ and}$
 $b \stackrel{def}{=} r := cu; done := TRUE$
- $eval(c) = \{(s, s) | s(i) < s(n)\}.$

Defining an algorithm by a set of actions (II)

Listing 4: PlusCal algorithm for summing a vector

```
—algorithm sumvect {  
variables  n=n0 , v=v0 , i=0 , cu=0 , r ;  
{  
w1: while ( i # n ) {  
    w2: cu:=cu+v[ i +1];  i:=i+1;  
};  
w3: r:=cu ; Gu  
}}
```

Global View



- Algorithm A is generated using EB2Algo

- $$\begin{array}{l} r : \in \mathbb{Z} \parallel i := 0 \parallel cu := 0 \parallel v := v0 \parallel n := n0 \\ \text{while } i < n \text{ do} \\ \quad i := i + 1 \parallel cu := cu + v(i + 1) \\ \text{od;} \end{array}$$

- Algorithm B is simply defined by

$$\text{guard}(r, i, cu, n, v, done) \equiv done = FALSE$$
$$\text{action} \equiv \left[\begin{array}{l} i = n \Rightarrow \left[\begin{array}{l} (r' = cu \wedge cu' = cu \wedge i' = i \wedge n' = n \\ \wedge v' = v \wedge done' = TRUE) \end{array} \right. \\ i < n \Rightarrow \left[\begin{array}{l} r' = r \wedge cu' = cu + v(i + 1) \wedge i' = i + 1 \wedge n' = n \\ \wedge v' = v \wedge done' = done \end{array} \right. \end{array} \right.$$

Algorithm C is generated using our transformations based on GKAT algebra

```
--algorithm sumvect {  
variables  n=n0,v=v0,i=0,cu=0,r;  
{  
w1:while (i # n) {  
  w2:cu:=cu+v[i+1]; i:=i+1;  
};  
w3:r:=cu;  
}}
```

From Event-B to TLA⁺

Current Summary

- 1 Overview
- 2 Programming by contract
 - Short view
 - The call-as-event idea
 - Main idea
- 3 Short Summary on Event-B
- 4 Internalizing events by merging
- 5 Summing a list of integer values
- 6 Summary - conclusion - future works

Summary of results

- Strategy for merging events
- Relating the last event and an operation call using GKAT
- Relating Event-B (analysis) and TLA^+ (validation): double checking using Event-B invariants for defining the inductive invariant of TLA^+

Points to address

- Missing systematic rules for deriving invariants.
- Automating the transformation of the final event into a PlusCal algorithm.
- More case studies ...

- Paradigm for planning refinements.
- Teaching why and how sequential algorithms are working.
- Atlas of *correct-by-construction* sequential algorithms

Happy birthday Egon

- Thank you for your involvement in the development of formal methods and for the interest you showed in each of them.

Happy birthday Egon

- Thank you for your involvement in the development of formal methods and for the interest you showed in each of them.
- Thank you for your insightful questions, which help us to make progress

Happy birthday Egon

- Thank you for your involvement in the development of formal methods and for the interest you showed in each of them.
- Thank you for your insightful questions, which help us to make progress
- Thank you for the passion you show

Happy birthday Egon

- Thank you for your involvement in the development of formal methods and for the interest you showed in each of them.
- Thank you for your insightful questions, which help us to make progress
- Thank you for the passion you show
- Thank you for your spirit that never gives up and pushes the boundaries

Happy birthday Egon

- Thank you for your involvement in the development of formal methods and for the interest you showed in each of them.
- Thank you for your insightful questions, which help us to make progress
- Thank you for the passion you show
- Thank you for your spirit that never gives up and pushes the boundaries
- Thanksssss to your wife who takes of you!